Layout Is Not a Feature

Why Universal Docking Fails Power Users and How Fixed Panel Architectures Eliminate the Cost

Registry: [HOWL-USER-1-2026]

Series Path: [HOWL-INFO-14-2026] → [HOWL-USER-1-2026]

DOI: 10.5281/zenodo.23938873

Date: November 2026

Domain: User Experience / Information Processing Theory

Status: Complete

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

---

1. The Operation

Every interaction between a user and a software interface is an operation. Clicking a button is one operation. Scanning a list of twelve items to find the correct one is one operation per item examined. Dragging a panel edge to resize it is one operation. Pressing a keyboard shortcut is one operation. Each operation is one irreducible transformation — one conscious act that the user performs to move from their current state toward their goal.

The operation is the unit of measurement for this paper. It is countable. It is observable. It does not depend on the software, the platform, or the domain. A click in a code editor and a click in a 3D modeling tool and a click in a spreadsheet are the same unit. The medium determines what the operation accomplishes. The cost to the user — one conscious act, one physical gesture, one moment of attention — is the same.

Operations are not free. Each one consumes three resources simultaneously. It consumes time: the user's working session is finite. It consumes attention: the user's conscious processing pipeline handles one operation at a time. It consumes physical effort: each mouse movement, click, or keystroke engages muscles in the hand, wrist, and forearm.

These three resources — time, attention, and physical effort — are the user's budget. The budget is fixed per session. It does not expand. Every operation spent on one task is an operation unavailable for another. The question for any interface design is therefore not "what can the user do?" but "what does the user spend their operations on?"

---

2. Productive and Non-Productive Operations

Not all operations advance the user's work. A developer editing code performs two kinds of operations. The first kind advances the work: reading a line of code, placing the cursor, typing a change, verifying the result. These are productive operations. They transform the artifact the user is building.

The second kind does not advance the work: clicking a tab to switch from the code editor to the file browser, scrolling a panel to find the right section, dragging a detached panel back into its dock, scanning a toolbar to locate a button. These are non-productive operations. They manage the tool. They are the cost of using the interface, not the cost of doing the work.

Every interface imposes both kinds. The goal of interface design is to minimize non-productive operations so that the user's budget is maximally available for productive work. A perfect interface would impose zero non-productive operations — every operation the user performs would advance their goal. No real interface achieves this, but the distance from zero is measurable and the mechanisms that create non-productive operations are identifiable.

---

3. Three Categories of Non-Productive Operations

Professional software interfaces built on the dockable panel paradigm — where panels can be dragged, detached, resized, tabbed, split, and rearranged by the user — impose non-productive operations in three categories.

3.1 Interleave An interleave operation switches the user's view between two pieces of information without advancing any task. The user is writing a database query and needs to check a column name. The column definitions are in a different tab than the query editor. The user clicks the column tab (one operation), reads the column name, clicks the query tab (one operation), and resumes typing. The two tab clicks advanced no work. They exist only because the interface cannot show both pieces of information simultaneously.

Interleave operations are the most common non-productive operations in tabbed and dockable interfaces. They arise whenever the user needs two pieces of information that the interface places in the same screen area behind different tabs. The user switches between them repeatedly, paying two operations per round trip, accomplishing nothing except changing which data is visible.

3.2 Cascade A cascade occurs when the interface's layout changes unexpectedly and the user must recover. The user is working in a code editor. Their mouse crosses a panel boundary during a rapid movement. A panel edge resizes, or a tab detaches into a floating window. The user did not intend this. The change disrupts their spatial model of the interface — their knowledge of where each panel is and what it contains.

Recovery from a cascade requires the user to notice the disruption (one operation), identify what changed (one to three operations), restore the layout (two to four operations — locating the displaced panel, dragging it back, verifying the result), and re-establish their spatial model (ongoing cost for the next thirty to sixty seconds as they consciously verify panel positions that were previously automatic). A single cascade costs six to nine explicit operations plus a degraded performance period during spatial model reconstruction.

Cascades are caused by drag-sensitive boundaries in the interface. Any pixel boundary that responds to mouse dragging is a potential cascade trigger. The probability of accidental triggering is proportional to the density of drag-sensitive boundaries and the total mouse movement per session.

3.3 Reduction A reduction operation occurs when the interface presents the user with a population of options and the user must find the correct one. The user selects an entity in a game engine. The properties panel shows all components attached to that entity: Transform, Renderer, Collider, Script, AudioSource, Animator, RigidBody, NavMeshAgent. The user needs the AudioSource component. They must scan the list (enumerate), skip irrelevant components (filter), and click the correct one (select). This is three to five operations that exist because the interface presented all options instead of the relevant one.

Reduction operations arise whenever a panel shows more content than the user needs for their current task. The cost scales with the number of irrelevant items displayed. A properties panel showing eight components when the user needs one imposes more reduction operations than a panel showing two.

---

4. Dissolution

Through repetition, operations become automatic. A new user of a code editor consciously locates the file browser panel, identifies it by its label, and clicks it. After weeks of daily use, the user's hand moves to the correct screen position without conscious thought. The operation of locating the file browser has dissolved — it still occurs, but it costs zero conscious effort. It has become structural, embedded in the user's practiced motor and spatial patterns.

Dissolution is the mechanism by which operation costs decrease toward zero. It applies to any operation that is performed repeatedly under consistent conditions. The user dissolves panel positions (knowing where each panel is without looking), navigation patterns (knowing which clicks reach a specific function), keyboard shortcuts (pressing key combinations without thinking about them), and interface sequences (performing multi-step workflows as a single fluid gesture).

Dissolution converts non-productive operations from a conscious cost to a structural free action. A dissolved panel-location operation costs zero attention, zero conscious processing time, and minimal physical effort (the motor pattern executes automatically). The user's attention budget is freed for productive work.

4.1 Validity Envelopes Every dissolved operation dissolved under specific conditions. The user dissolved the file browser's position under the condition that the file browser is on the left side of the screen, in the first panel slot, at approximately 250 pixels wide. These conditions define a region — the validity envelope — within which the dissolution holds.

Inside the envelope, the dissolved operation costs zero. Outside the envelope, the operation promotes back to conscious processing and costs operations again. The user must re-locate the panel, re-identify it, and re-learn its position. This promotion is not instantaneous — the user first experiences a moment of confusion (the panel is not where they expected it), then must actively search for it.

The width of a validity envelope depends on the variability of conditions during dissolution. A user who always works at the same resolution, on the same monitor, with the same layout has narrow envelopes. A user who works across multiple monitors and resolutions has wider envelopes. Narrow envelopes break more easily.

4.2 Cascade as Envelope Violation A cascade, defined in Section 3.2, is now precisely characterized as the simultaneous violation of multiple validity envelopes. When a panel detaches accidentally, the validity envelopes for that panel's position, its neighboring panels' sizes, and the overall spatial model are all violated at once. Multiple dissolved operations promote to conscious processing simultaneously.

The severity of a cascade is determined by how many dissolved operations promote at once, not by the magnitude of the triggering event. A two-pixel mouse slip that detaches a panel is a tiny event. The cascade it causes — three to five dissolved spatial assumptions all promoting at once — is large. A loud noise that causes the user to flinch but does not change the layout causes zero cascade because no validity envelopes are violated. Cascade severity is a property of the interface's fragility topology, not of the triggering event.

---

5. Counting Operations in Practice

This section counts operations for five professional applications, comparing the current dockable-panel implementation against a fixed-panel architecture. The fixed-panel architecture uses six named panel areas at fixed screen positions: a tree panel (left), a canvas (center), a properties panel (right top), a view panel (right bottom), a tool set bar (bottom, toggleable), and a tray (bottom of canvas, toggleable). Each area accepts arbitrary content determined by the active tool. Browser tabs at the top provide independent workspaces, each carrying its own complete panel state.

5.1 Code Editor Task: Find a symbol's definition, read its properties, edit it.

Current (VS Code): The user presses a keyboard shortcut to go to the definition (one operation). If multiple results appear, the user scans the results list and selects the correct one (two operations). The file opens in the editor area. The user reads the function signature in the editor (one to two operations). To see file metadata or symbol properties,

the user must open a different panel or hover over the symbol (one operation — a context switch away from the code). Total: four to six operations.

Fixed-panel: The user navigates the tree panel or uses a search command (one to two operations). The symbol's content appears in the canvas. The properties panel simultaneously shows the symbol's metadata, type information, and references without any additional action — it is already visible. The view panel shows the file path confirming the location. Total: two to three operations.

Savings: two to three operations per occurrence. The savings come from eliminating the context-switch operation required to view metadata. In the dockable system, code and metadata occupy the same screen area behind different views. In the fixed-panel system, they are always simultaneously visible in different panel areas.

Task: Recover from accidental panel detachment.

Current (VS Code): Notice panel is missing (one operation). Identify which panel detached (one operation). Find the floating window on screen or taskbar (one to three operations). Drag the panel back to the correct dock position (two to three operations). Verify the layout is restored correctly (one operation). Total: six to nine operations.

Fixed-panel: Cannot occur. Panels do not detach. Total: zero operations.

Savings: six to nine operations per incident. The entire category of cascade recovery is eliminated.

5.2 3D Modeling Application Task: Select an object, modify a property, see the result in the viewport.

Current (Blender): Click the object in the viewport or outliner (one operation). The properties panel updates automatically. Locate the correct properties tab among Object, Modifiers, Material, Constraints, Particles, Physics, and others (one operation to enumerate, one to select). Scroll to the property section (one to two operations). Edit the value (one operation). The viewport updates. Total: four to six operations.

Fixed-panel: Click the object in the canvas or tree panel (one operation). The properties panel shows properties filtered by the active tool domain — the tool selection has already determined which property category is visible (zero operations for tab location). Edit the value (one operation). The canvas updates. Total: two to three operations.

Savings: two to three operations per edit cycle. The savings come from eliminating the tab enumeration step. Blender's properties panel contains seven to ten tabs. The user must run a reduction pipeline (enumerate tabs, filter to relevant, select) on every object selection. The fixed-panel system pre-computes this reduction through the tool domain system.

5.3 Database Administration Tool Task: Find a table, examine its columns, write a query, view results.

Current (DataGrip): Expand the database tree to find the table (three to five operations for expanding server, database, schema, tables). Click the table (one operation). The table's DDL appears in a tab in the editor area, replacing the previous editor tab. The user notices the query editor tab is now behind the DDL tab (one operation). Click the query editor tab (one operation — interleave). Write the query referencing column names (one to three operations). To check column names again, switch back to the DDL tab (one operation — interleave). Execute the query (one operation). Results appear in a bottom panel. Total: nine to thirteen operations.

Fixed-panel: Navigate the tree panel to the table (three to five operations — same depth). The properties panel shows column definitions simultaneously (zero operations — already visible). The canvas contains the query editor (zero operations — already visible). Write the query while reading column names from the properties panel (zero interleave operations — both visible). Execute the query (one operation). Results appear in the canvas or tray. Total: five to seven operations.

Savings: four to six operations per query cycle. Nearly all savings come from eliminating interleave. The dockable system forces column definitions and the query editor to compete for the same screen area. The user switches between them repeatedly. The fixed-panel system places them in different panel areas that are always simultaneously visible.

5.4 Email Client Task: Write a reply while referencing an attachment from a different message.

Current (Outlook): Open reply compose window (one operation). The reply window covers or replaces the reading pane. Need to reference an attachment from a different message: minimize the compose window or open it in a new window (two to three operations — layout management). Find the other message (one to two operations). Open the attachment (one operation). Switch back to the compose window (one operation — interleave). Refer to the attachment while typing, switching back and forth (one operation per reference). For three references: three interleave operations. Total: ten to thirteen operations.

Fixed-panel: Reply compose opens in the canvas (one operation). The original message thread remains visible in the properties or view panel (zero operations — already visible). Navigate the tree to find the other message (one to two operations). Attachment preview opens in a canvas sub-panel alongside the compose area (one operation to initiate the split). Refer to the attachment while typing (zero interleave operations — both visible). Total: three to five operations.

Savings: seven to eight operations per composition. The savings are entirely from eliminating interleave between the compose surface and reference material. The dockable system forces the user to manage window arrangement during the act of writing, which is a layout management task that has nothing to do with composing the message.

5.5 Game Engine Editor Task: Edit a tile layer while inspecting entity data.

Current (Unity): Open the tile palette window if not already open (one operation). Position the tile palette window so it does not cover the scene view (two to three operations if first time). Select a tile layer (one operation). Paint tiles in the scene view (variable — direct manipulation). Need to check entity data: click the entity in the hierarchy (one operation). The Inspector switches from tile palette settings to entity components (automatic, but the tile palette context is lost). Read entity data (zero operations). Switch back to the tile palette: click the tile palette window or re-select the tilemap (one to two operations — interleave). Total per context switch: two to three operations, with cascade risk from Inspector context loss.

Fixed-panel: The tool set bar is set to the Tile Painting tool. The tool set bar shows painting controls. The canvas is pinned to the game scene via the canvas override toggle. The side panels are split to the Data Browser tool. The tree panel shows the data navigation. The properties panel shows entity properties. The tray shows tile painting controls within the canvas area. All visible simultaneously. To inspect an entity while painting, the user navigates the tree panel (one to two operations). Entity properties appear in the properties panel (zero operations). Tile painting controls remain visible in the tray (zero operations — no context loss). Total: one to two operations, zero interleave, zero cascade.

Savings: one to four operations per context switch, plus complete cascade elimination. The three-axis model (tool set bar on one axis, side panels on a second axis, canvas on a third axis) eliminates the mutual exclusion that forces Unity's Inspector to serve one context at a time.

---

6. Scaling to a Working Day

A power user performing concentrated professional work completes two hundred to four hundred task cycles per eight-hour session. A task cycle is one complete unit of work: locate the target, read it, change it, verify the change. The rate varies by domain — a developer commits six to ten meaningful edits per minute during active coding, a 3D artist makes four to eight manipulations per minute during modeling, a database administrator runs two to four query-edit-verify cycles per minute during schema work.

Across five to six hours of active work within an eight-hour day, these cycles accumulate to thousands of individual operations. The non-productive operations identified in Section 5 occur at rates proportional to the task cycle count.

6.1 Daily Interleave Tax From the operation counts in Section 5, dockable interfaces impose one to four interleave operations per task cycle. At a conservative average of two interleave operations per cycle, a power user performing three hundred task cycles per day executes six hundred interleave operations daily. These operations advance no work. They switch views, click tabs, and re-locate information that was visible a moment ago.

In the fixed-panel system, interleave operations are zero. The information the user needs is simultaneously visible across different panel areas. The daily savings: six hundred operations.

6.2 Daily Cascade Tax Accidental layout disruptions occur at a rate determined by the density of drag-sensitive boundaries in the interface and the user's total mouse movement. A power user moving the mouse ten thousand to fifteen thousand times per session in an interface where thirty to forty percent of panel boundaries are drag-sensitive will experience two to five accidental layout disruptions per session.

Each disruption costs six to nine recovery operations plus a degraded performance period of thirty to ninety seconds. Daily cascade cost: twelve to forty-five explicit recovery operations plus two to eight minutes of degraded throughput.

In the fixed-panel system, the cascade rate is zero because no drag-sensitive layout boundaries exist. Panels cannot be detached, resized accidentally, or rearranged.

6.3 Daily Reduction Tax Every time a dockable interface presents the user with a panel containing all possible options for all possible contexts, the user must run a reduction pipeline to find the relevant content. In a game engine editor, selecting an entity shows all components. In a 3D modeling tool, the properties panel shows all property categories. In a database tool, the toolbar shows all available actions.

At three reduction operations per entity selection and fifty to one hundred selections per session, the reduction tax is one hundred fifty to three hundred operations per day. The fixed-panel system's tool domain pre-selection reduces this to zero to one operation per selection.

6.4 Daily Total

Cost Category	Dockable UI (daily)	Fixed-Panel (daily)
Interleave operations	600	0
Cascade recovery operations	12-45	0
Reduction operations	150-300	0-50
Total non-productive operations	**762-945**	**0-50**

The fixed-panel system eliminates seven hundred to nine hundred non-productive operations per day.

---

7. Flow State

Flow state is the condition in which the user's entire conscious processing pipeline is occupied by productive work. All supporting operations — panel location, navigation, tool management — have dissolved to zero cost. The user is not thinking about the interface. Every operation in their pipeline is domain work: writing code, shaping geometry, composing queries, editing text.

Flow state is the highest-throughput condition a user can achieve. During flow, the user operates at their maximum productive rate. No attention is diverted to tool management.

No operations are wasted on interleave, cascade recovery, or reduction. The user's full budget is available for the work itself.

7.1 Entry Cost Entering flow state requires dissolving all supporting operations for the current session. The user must stop consciously thinking about where panels are, which tab to click, how to navigate, and where to look for information. This typically takes ten to twenty minutes at the start of a work session as the user re-establishes their spatial model and working context.

In a dockable interface, entry cost includes re-verifying layout knowledge. The user opens the application and must confirm that panels are where they left them. If a software update rearranged defaults, or the window was resized, or a workspace preset loaded differently, some dissolved layout assumptions have been invalidated. With twenty-five to fifty dissolved layout elements per application, the re-verification pass consumes two to five minutes before the user stops thinking about the interface.

In the fixed-panel system, layout re-verification costs zero operations. The tree panel is on the left. It was on the left in the previous session. It will be on the left after every software update. The user never re-verifies panel positions. Flow state entry cost for layout is zero.

7.2 Disruption Surface Flow state is not a binary condition that persists once entered. It is a continuous condition that breaks when any dissolved operation promotes back to conscious processing. During flow, the user operates with maximum dissolved inventory — maximum fragility. Any disruption that promotes even one supporting operation breaks flow because the conscious pipeline, which was fully occupied by productive work, must allocate capacity to the promoted operation.

Dockable interfaces create a permanent disruption surface through three mechanisms:

Drag-sensitive boundaries. Any mouse movement near a panel edge can trigger an accidental resize or detachment. The user's dissolved spatial model promotes. They must stop productive work, assess the damage, restore the layout, and wait for the spatial model to re-dissolve. Flow recovery time: two to five minutes.

Tab proliferation. Opening new content adds tabs to existing tab bars. The user's dissolved tab-order model (which position holds which content) is invalidated. The next tab-by-position reach grabs the wrong content. Flow disruption: five to fifteen seconds per incident, compounding across dozens of incidents per session.

Panel context switching. Clicking an element causes a shared panel (such as the Inspector or Properties panel) to replace its content. If the user was reading that panel when the content changed, their reading operation is interrupted. Flow disruption: two to five seconds per incident.

In the fixed-panel system, drag-sensitive layout boundaries do not exist. Tab proliferation within panel areas does not exist — panel content is determined by tool commands, not by user-opened tabs. Panel context switching still occurs (clicking an entity updates the properties panel) but is consistent and predictable because the tool domain pre-filters the content, accelerating re-dissolution of the content-switching pattern.

7.3 Flow Duration and Duty Cycle Flow duration is the time between entry and the next involuntary disruption. It is bounded by the minimum interval between disruptions.

In a dockable interface, involuntary UI disruptions include accidental layout events (two to five per session), tab proliferation incidents (ten to twenty per session), and panel context losses (fifteen to thirty per session). Across a six-hour active work period, that is twenty-seven to fifty-five involuntary disruptions, or one every seven to thirteen minutes.

Flow re-entry after a disruption takes two to five minutes. If disruptions occur every seven to thirteen minutes and re-entry takes two to five minutes, the user achieves sustained flow for only two to eight minutes at a time. The flow duty cycle — the fraction of active work time spent in flow — is:

(disruption interval − re-entry time) ÷ disruption interval

For dockable interfaces: (7 to 13 − 2 to 5) ÷ (7 to 13) = 38% to 85%, with a midpoint of approximately 60%.

In the fixed-panel system, all UI-sourced disruptions are eliminated. The remaining disruptions come from external sources: messaging notifications, phone calls, colleague interruptions. Assuming external disruptions occur every twenty to thirty minutes (typical for an open-office environment):

(20 to 30 − 2 to 5) ÷ (20 to 30) = 60% to 93%, with a midpoint of approximately 75%.

The fixed-panel system raises the flow duty cycle from approximately 60% to approximately 75% by eliminating UI-sourced disruptions entirely. The improvement is fifteen to twenty-five percentage points.

7.4 Session Length Effect Longer flow sessions operate at higher efficiency than shorter ones. Each flow session has a ramp-up period where the user rebuilds mental context before reaching peak throughput. A five-minute flow session with a two-minute ramp produces three minutes of peak work (60% within-session efficiency). A twenty-minute flow session with the same two-minute ramp produces eighteen minutes of peak work (90% within-session efficiency).

The fixed-panel system's flow sessions are two to three times longer than those in dockable systems. Each session operates at higher within-session efficiency. The compounding effect: more total flow minutes, each minute at higher efficiency.

7.5 Annual Impact Over two hundred fifty working days per year, at fifty-four additional minutes of flow per day (the difference between 60% and 75% duty cycle applied to six hours of active work):

225 additional hours of flow-state productive work per year.

Each of these hours is at higher within-session efficiency due to longer unbroken sessions. Over a decade: 2,250 additional hours of flow-state work.

---

8. The Dissolution Budget

Every user has a finite capacity for dissolution. The number of operations that can be dissolved is bounded by the repetition opportunities available in the user's working life and the cognitive capacity for procedural learning. This capacity is not unlimited. Every operation that dissolves to zero cost consumed repetitions during the dissolution period. Every dissolved operation occupies a slot in the user's procedural memory.

Layout management knowledge competes with domain knowledge for this finite budget. A user of a dockable interface must dissolve the following per application:

- Panel positions: five to eight dissolved spatial locations
- Tab-to-function mappings: ten to twenty dissolved associations (which tab contains which function)
- Keyboard shortcut-to-panel mappings: five to fifteen dissolved shortcut associations
- Layout recovery procedures: one to three dissolved recovery sequences (how to reset or restore the layout)
- Workspace preset-to-task mappings: two to five dissolved associations (which workspace preset to activate for which task type)
- Rearrangement procedures: one to three dissolved sequences (how to split, merge, or rearrange panels for a new task)

Total dissolved layout management operations per application: twenty-five to fifty.

A power user working across three to four applications (an IDE, a 3D editor, a database tool, and a browser) maintains seventy-five to two hundred dissolved layout management operations. These are not productive knowledge. They are knowledge about how to operate tools, not how to do work. They were purchased with repetitions that could have been spent dissolving domain operations. They occupy dissolution capacity that could hold domain expertise.

In the fixed-panel system, dissolved layout management operations per application approach zero:

- Panel positions: zero (fixed, nothing to dissolve — the positions are structural facts, not learned associations)
- Tab-to-function mappings: zero (no panel tabs — tool commands determine content)
- Recovery procedures: zero (nothing breaks)
- Workspace presets: zero (the three-axis model replaces workspace presets with tool selection)
- Rearrangement procedures: zero (no rearrangement possible or needed)

The only dissolved operations are the tool domain meanings (which tool shows what content) and keyboard shortcuts for tool switching. These are productive knowledge — they map directly to the work domain (selecting the Tile Painting tool, switching to the Data Browser) rather than to the UI management domain.

8.1 Cross-Application Consistency If every application used the fixed-panel model, the user would dissolve one spatial layout — six panel areas at fixed positions — and apply it to every application they use. The dissolved operation "the properties panel is on the upper right" would be valid across the code editor, the 3D modeling tool, the database administrator, the email client, and the game engine editor. One dissolution serves all applications.

In the dockable paradigm, each application has a different layout model. VS Code's panel arrangement conventions differ from Blender's, which differ from Unity's, which differ from DataGrip's. The user maintains a separate dissolved spatial model per application. Switching applications requires a context switch not just in the work domain but in the layout domain — the user must recall which application places panels where. This context switch costs one to three operations if dissolved (subconscious adjustment) or five to ten operations if not fully dissolved (conscious re-orientation).

The fixed-panel model eliminates cross-application layout context switches entirely. The spatial model is the same everywhere. This frees dissolution capacity for the per-application domain knowledge that actually differs between tools.

---

9. Physical Cost

Every non-productive operation is a physical action. A mouse click engages the finger flexor muscles for fifty to one hundred milliseconds. A mouse movement engages the wrist extensors and forearm pronator for the duration of the movement. A drag operation — pressing the mouse button, holding it, moving the mouse, and releasing — engages sustained muscle contraction for five hundred to two thousand milliseconds, the highest-strain mouse gesture in common use.

The daily non-productive operation counts from Section 6 translate directly to physical actions:

9.1 Daily Physical Actions Eliminated

Action Type	Dockable UI (daily)	Fixed-Panel (daily)	Eliminated
Interleave clicks	600	0	600
Interleave mouse movements (2 per click)	1,200	0	1,200

Action Type	Dockable UI (daily)	Fixed-Panel (daily)	Eliminated
Cascade recovery drags	10-15	0	10-15
Reduction scan eye movements	150-300	0-50	100-300
Total unnecessary physical actions	**1,960-2,115**	**0-50**	**1,910-2,115**

9.2 Annual and Career Scale Over two hundred fifty working days per year:

Metric	Per Year	Per Decade
Unnecessary mouse movements eliminated	300,000-400,000	3,000,000-4,000,000
Unnecessary clicks eliminated	150,000-225,000	1,500,000-2,250,000
Sustained-pressure drags eliminated	2,500-3,750	25,000-37,500

Cumulative repetitive strain is proportional to total movement count and the force profile of each movement. Sustained-pressure drags are disproportionately harmful because they require simultaneous contraction and movement — the wrist extensors stabilize while the forearm moves, creating shear stress on the tendons. Eliminating twenty-five thousand to thirty-seven thousand sustained-pressure drags over a decade removes a measurable RSI risk factor.

The interleave mouse movements are individually low-strain but accumulate. Three to four million unnecessary mouse movements over a decade is a non-trivial load on the wrist extensor group, particularly for users already at elevated RSI risk from their productive work.

---

10. The Fixed-Panel Architecture

The previous sections established that dockable interfaces impose measurable costs: seven hundred to nine hundred non-productive operations per day, a fifteen to twenty-five percentage point reduction in flow duty cycle, seventy-five to two hundred dissolved layout management operations per user, and nearly two thousand unnecessary physical actions per day. This section presents a fixed-panel architecture that eliminates these costs and demonstrates that it can represent any application.

10.1 Six Panel Areas The architecture defines six named panel areas at fixed screen positions:

Area	Position	Default Purpose
Tree	Left	Navigation: file trees, hierarchy lists, record lists, breadcrumbs
Canvas	Center	Primary workspace: code editing, 3D viewport, document editing, game scene
Properties	Right, upper	Contextual detail: selected item properties, field editors, metadata
View	Right, lower	Supplementary context: path info, breadcrumbs, status, secondary data
Tool Set	Bottom, full width, toggleable	Tool-specific controls: palettes, timelines, terminals, build output
Tray	Bottom of canvas, toggleable	In-context controls: painting tools, transport controls, inline actions

Browser tabs run along the top of the screen. Each tab carries its own independent state for all six panel areas, providing the equivalent of multiple independent workspaces.

10.2 Panel Areas Are Content-Agnostic The names describe default purpose and screen position, not capability. Each panel area is a rectangular clip region that accepts an arbitrary tree of UI elements. Any panel area can contain any content: text editors, 3D viewports, data grids, node graphs, image surfaces, terminal emulators, or custom render targets. Any panel area can be subdivided internally into any arrangement of sub-panels.

The tree panel can contain two side-by-side trees. The canvas can contain four viewports in a grid. The tool set area can contain three panels showing different data. The properties panel can contain a document editor. The content is unbounded. The positions are fixed.

10.3 Three-Axis Independence Each browser tab controls three independent axes that determine which tool's content appears in which panel areas:

Axis	Controls	Purpose
Tool Set	Tool set bar content, and side panels when unified	Primary tool selection
Side Panel	Tree, properties, and view content when split	Independent data inspection alongside the primary tool

Axis	Controls	Purpose
Canvas	Canvas and tray content	Independent viewport or workspace

When all three axes point to the same tool, the user sees a unified single-tool view. When the axes point to different tools, the user sees content from multiple tools simultaneously without any interleave cost. The tile painter's controls appear in the tool set bar while the data browser's navigation and properties appear in the side panels while the game scene renders in the canvas. All three are visible. No tab switching. No interleave operations.

This three-axis model replaces workspace presets. Instead of saving and loading entire screen arrangements, the user selects which tools are active on each axis. The layout is always the same. The content changes.

10.4 Tool Registration Tools register as data records, not as code modules. Each tool record contains text fields that name commands for each panel area. The system dispatches these commands each frame to populate the panel areas with content. Adding a new tool means adding a data record — no code changes to the frame loop, no new panel types, no layout modifications.

Empty command fields mean the tool does not populate that panel area. A tool that only needs a tree and properties panel leaves the canvas, tray, tool set, and view command fields empty. The panel areas without commands simply show no content. The layout does not change.

10.5 Universality Section 10.2 established that each panel area accepts arbitrary content. This means any application's interface can be mapped onto the six areas by assigning its components to the areas whose screen positions match their function. This is not a claim that the mapping is always obvious or that no design work is required. It is a structural claim: six content-agnostic rectangular regions at fixed positions — left, center, right-top, right-bottom, full-bottom, and center-bottom — can host any combination of UI elements that any application requires, because they impose no constraints on content.

The following mappings are representative, not exhaustive:

A code editor places the file tree in the tree panel, the editor surface in the canvas, symbol metadata in properties, file path breadcrumbs in view, and the integrated terminal in the tool set bar. A 3D modeling application places the scene outliner in the tree panel, the 3D viewport in the canvas, the object properties in properties, the transform display in view, and the timeline in the tool set bar. A database tool places the schema browser in the tree panel, the query editor and results in the canvas, the table column definitions in properties, the connection info in view, and the query history in the tool set bar. A music production application places the track list in the tree panel, the arrangement view in the canvas, the channel strip in properties, the level meters in view, and the mixer in the tool set bar.

Each mapping follows the same structural principle: navigation on the left, primary workspace in the center, contextual detail on the right, extended tools on the bottom. The principle dissolves once for all applications. The user learns it once and applies it everywhere.

---

11. The Abdication Diagnosis

The dockable panel paradigm emerged from a specific organizational constraint: the design team cannot predict which workflows their users will need. An engine that serves mobile game developers, film studios, architectural visualization, and VR experiences cannot ship a single layout that works for all of them. The docking system is the response: give every user the ability to build their own layout.

This response transfers the cost of layout design from the team that built the software to every individual user of the software. The team performs the design work zero times. Each user performs it independently. If the software has one hundred thousand users, the layout design work is performed one hundred thousand times instead of once.

The transfer is framed as a feature. Marketing materials describe it as "customization," "flexibility," and "workflow optimization." Documentation explains how to drag panels, save workspaces, and reset layouts. Tutorials demonstrate the power of arranging the interface to suit personal preferences.

But the work being transferred is not productive work. It is non-productive overhead. The user does not want to design a panel layout. The user wants to edit code, model geometry, compose queries, or build a game. Layout design is an obstacle between the user and their goal. Giving the user better tools to overcome the obstacle is not the same as removing the obstacle.

The fixed-panel architecture removes the obstacle. The layout is designed once by the team that built the software. The design encodes the structural insight that navigation belongs on the left, the primary workspace belongs in the center, contextual detail belongs on the right, and extended tools belong on the bottom. This insight applies to every application domain. The user receives the finished design and proceeds directly to productive work.

The objection is immediate: what if the fixed layout is wrong for a specific user? The answer is that the layout is not the content. The user does not need to rearrange rectangular regions on screen. The user needs to see the right content in the right places. The three-axis model provides this by letting the user select which tools populate each panel group. The layout remains fixed. The content adapts to the task.

The remaining objection is panel sizing: sometimes the tree needs to be wider, sometimes the canvas needs to be larger. Draggable edges between fixed panel areas address this need — the only capability from the docking paradigm that solves a real continuous-adjustment problem. The edges are between fixed areas, not between arbitrary dockable panels. They

cannot detach panels, create floating windows, or trigger cascades. They adjust size ratios within a stable structure.

---

12. Conclusion

Dockable panel systems impose a measurable tax on professional software users. The tax has three components: interleave operations that switch views without advancing work, cascade operations that recover from accidental layout disruption, and reduction operations that scan overcrowded panels to find relevant content. These non-productive operations total seven hundred to nine hundred per day for a power user.

The tax reduces flow state duty cycle by fifteen to twenty-five percentage points, shortens individual flow sessions by a factor of two to three, and costs approximately two hundred twenty-five hours of flow-state productive work per year. Over a decade, the cumulative loss is two thousand two hundred fifty hours.

The tax consumes dissolution capacity. Each user maintains twenty-five to fifty dissolved layout management operations per application, totaling seventy-five to two hundred across their tool chain. This capacity is unavailable for domain knowledge dissolution, slowing the development of productive expertise.

The tax imposes physical strain. Approximately two thousand unnecessary physical actions per day, including sustained-pressure drag operations, accumulate to millions of unnecessary mouse movements over a career, contributing to repetitive strain injury risk.

A fixed-panel architecture with six content-agnostic areas at stable screen positions eliminates the tax entirely. The architecture can represent any application because the panel areas impose no constraints on content. The three-axis tool selection model replaces workspace presets with tool choices, providing content flexibility without layout instability. Draggable edges between fixed areas provide the one continuously useful capability from the docking paradigm — size adjustment — without the cascade risk.

Layout management is not a feature. It is a cost. The dockable paradigm charges this cost to every user, every day, for the lifetime of their use of the software. The fixed-panel architecture pays it once, at design time, and the user pays zero.

References

[HOWL-INFO-14-2026] — Bits and Ops: A Complete Theory of Information. The theoretical foundation for operation counting, dissolution, validity envelopes, cascade severity, processing entropy, and the fundamental inequality.

[HOWL-INFO-11-2026] — Zero, One, and Infinity. Three cardinalities as intrinsic properties. Foundation for the reduction pipeline.

[HOWL-INFO-12-2026] — Reduction to Cardinality One. The four-stage reduction pipeline: enumerate, filter, score, select.

[HOWL-INFO-13-2026] — Six States / Mathematical Theory / Measurement Theory. Dissolution, processing entropy, the op as fundamental unit.

[HOWL-MATH-16-2026] — Geometry of Dissolution and Fragility. Validity envelopes, cascade severity function, fragility topology.

[HOWL-MATH-18-2026] — Concurrency Tax from System Structure. Interleave as a tax component. Contention graph motifs.

Appendix Tables

Table A: Operation Classification Taxonomy

Classification	Definition	Advances Work	Consumes Budget	Example	Elimination Method
Productive	Transforms the artifact the user is building	Yes	Yes	Typing code, adjusting a vertex, editing a cell value	Cannot eliminate — this is the work
Interleave	Switches view without advancing any task	No	Yes	Clicking a tab to see column names, then clicking back	Simultaneous visibility across fixed panel areas
Cascade Recovery	Restores layout after accidental disruption	No	Yes	Dragging a detached panel back to its dock position	Eliminate drag-sensitive layout boundaries
Reduction	Scans a population of options to find the relevant one	No	Yes	Scanning eight component tabs to find AudioSource	Pre-compute reduction through tool domain selection

Classification	Definition	Advances Work	Consumes Budget	Example	Elimination Method
Re-verification	Confirms a dissolved assumption still holds	No	Yes	Checking that the tree panel is still on the left after an update	Fixed positions that never change
Re-dissolution	Re-practicing a promoted operation until it dissolves again	No	Yes	Rebuilding muscle memory for a panel location after layout change	Prevent the promotion that triggered re-dissolution

Table B: Cascade Trigger Inventory for Dockable Interfaces

Trigger	Physical Cause	Typical Frequency (per session)	Dissolved Elements Promoted	Recovery Ops	Recovery Time (seconds)	Flow Break Duration (minutes)
Panel edge drag	Mouse crosses resize boundary during rapid movement	1-3	2-3 (panel sizes, spatial model)	3-5	5-15	2-3
Tab detach	Mouse drags tab past detach threshold	0.5-2	3-5 (panel position, tab order, spatial model, available screen area)	6-9	15-45	3-5

Trigger	Physical Cause	Typical Frequency (per session)	Dissolved Elements Promoted	Recovery Ops	Recovery Time (seconds)	Flow Break Duration (minutes)
Splitter shift	Mouse depresses on splitter bar during transit	0.5-1	2-4 (relative panel sizes, content visibility, scroll positions)	3-6	10-20	2-3
Workspace preset load	User selects wrong preset or preset loads differently than expected	0-0.5	10-20 (entire spatial model)	10-15	30-90	5-10
Software update layout reset	Application updates and resets to default layout	0-0.01 (per session, periodic)	20-50 (entire dissolved layout inventory for this application)	15-30	120-600	10-30
Monitor change	User connects external monitor, resolution changes	0-0.01 (per session, periodic)	10-30 (all position-dependent dissolutions)	10-20	60-300	5-15
Colleague use	Another person used the workstation and saved a different layout	0-0.01 (per session, periodic)	20-50 (entire spatial model)	15-30	120-600	10-30

Table C: Drag-Sensitive Boundary Density by Application

Application	Total Panel Boundary Pixels (1920 × 1080)	Drag-Sensitive Boundary Pixels	Drag-Sensitive Percentage	Detachable Tab Bar Pixels	Total Cascade Surface
VS Code (default layout)	~4,200	~2,800	67%	~1,200	~4,000
Blender (default layout)	~6,400	~4,500	70%	~800	~5,300
Unity (default layout)	~5,100	~3,200	63%	~2,400	~5,600
Unreal (default layout)	~5,800	~3,600	62%	~2,800	~6,400
DataGrip (default layout)	~3,800	~2,200	58%	~1,600	~3,800
Fixed-panel system	~3,200	0	0%	0	0

Table D: Interleave Operations Per Task Cycle by Application

Application	Task Cycle	Information Pair Requiring Interleave	Interleave Ops (Dockable)	Interleave Ops (Fixed-Panel)	Savings
Code Editor	Edit function referencing another file	Source file ↔ reference file	2	0 (canvas split)	2
Code Editor	Debug with variable inspection	Code editor ↔ variables panel	2	0 (props shows variables)	2

Application	Task Cycle	Information Pair Requiring Interleave	Interleave Ops (Dockable)	Interleave Ops (Fixed-Panel)	Savings
Code Editor	Write code while reading documentation	Editor tab ↔ docs tab	2	0 (canvas split)	2
3D Modeling	Adjust material while viewing result	Material editor ↔ viewport	2	0 (props + canvas simultaneous)	2
3D Modeling	Edit UV while checking texture	UV editor ↔ texture preview	2	0 (canvas split or view panel)	2
Database	Write query referencing column names	Query tab ↔ DDL tab	2	0 (canvas + props simultaneous)	2
Database	Compare query results with expected output	Results tab ↔ reference tab	2	0 (canvas split)	2
Email	Compose reply referencing attachment	Compose window ↔ attachment viewer	2-4	0 (canvas split)	2-4
Game Editor	Paint tiles while checking entity data	Tile palette ↔ Inspector	2-3	0 (three-axis separation)	2-3
Game Editor	Edit scene actor while viewing game	Inspector ↔ game view tab	1-2	0 (props + canvas simultaneous)	1-2
Spreadsheet	Edit formula referencing another sheet	Cell editor ↔ reference sheet tab	2	0 (canvas split)	2

Application	Task Cycle	Information Pair Requiring Interleave	Interleave Ops (Dockable)	Interleave Ops (Fixed-Panel)	Savings
Video Editor	Trim clip while monitoring output	Source monitor ↔ program monitor	2	0 (canvas split)	2

Table E: Reduction Pipeline Cost by Panel Type

Panel Type	Total Options Visible	Relevant Options Per Task	Enumerate Ops	Filter Ops	Score Ops	Select Ops	Total Reduction Ops	Fixed-Panel Reduction Ops
Unity Inspector (all components)	6-12 components	1	1	1-2	1	1	4-5	0 (domain preselects)
Blender Properties (all tabs)	7-10 tabs	1	1	1	1	1	4	0 (domain preselects)
VS Code Activity Bar (all panels)	5-8 icons	1	1	1	0	1	3	0 (tree always visible)
DataGrip toolbar (all actions)	15-25 buttons	1-2	1	1-2	1	1	4-5	0-1 (tool set shows relevant actions)

Panel Type	Total Options Visible	Relevant Options Per Task	Enumerate Ops	Filter Ops	Score Ops	Select Ops	Total Reduction Ops	Fixed-Panel Reduction Ops
Unreal Details panel (all categories)	8-15 categories	1-2	1	1-2	1	1	4-5	0 (domain pre-selects)
Photoshop Layers + Channels + Paths tabs	3 tabs	1	1	0	0	1	2	0 (always visible in props)

Table F: Dissolved Layout Elements Inventory by Application

Application	Panel Positions	Tab Mappings	Shortcuts	Recovery Procedures	Workspace Presets	Rearrangement Sequences	Total
VS Code	5	8-12	10-15	1-2	2-3	1-2	27-39
Blender	8	10-15	15-20	2-3	3-5	2-3	40-54
Unity	6	12-18	8-12	1-2	2-4	1-3	30-45
Unreal	7	15-20	10-15	2-3	3-5	2-3	39-53
DataGrip	5	6-10	5-8	1	1-2	1-2	19-28
Photoshop	6	8-12	12-18	1-2	3-5	2-3	32-46
Fixed-panel system (any app)	0	0	3-8	0	0	0	3-8

Table G: Cross-Application Context Switch Cost

Transition	Dissolved Spatial Models Affected	Layout Context Switch Ops (Dockable)	Layout Context Switch Ops (Fixed-Panel)	Domain Context Switch Ops (Both)
VS Code → Blender	2 (exit VS Code model, enter Blender model)	3-5	0	5-10 (domain-specific)
Blender → Unity	2	3-5	0	3-8
Unity → DataGrip	2	2-4	0	4-8
DataGrip → VS Code	2	2-4	0	3-6
Any → Any (fixed-panel)	0	0	0	Domain ops only

A power user switching between three applications ten times per day pays thirty to fifty layout context switch ops daily in the dockable paradigm and zero in the fixed-panel system. Annual cost: seven thousand five hundred to twelve thousand five hundred unnecessary operations.

Table H: Validity Envelope Dimensions for Layout Dissolution

Dissolved Element	Envelope Dimension	Typical Width	Common Violation Cause	Promotion Cost (ops)
Panel X position	Screen resolution	Narrow (exact pixel range)	Monitor change, window resize	2-3
Panel Y position	Screen resolution	Narrow	Monitor change, window resize	2-3
Panel width	Window width	Medium (proportional)	Window resize, splitter drag	1-2
Panel height	Window height	Medium	Window resize, splitter drag	1-2
Tab order within panel	Tab insertion/removal	Narrow (exact sequence)	Opening new tab, closing tab, plugin adding tab	2-4

Dissolved Element	Envelope Dimension	Typical Width	Common Violation Cause	Promotion Cost (ops)
Tab label content	Application state	Wide (usually stable)	Rare — tab labels seldom change	1
Panel visibility	Application state	Medium	Workspace switch, toggle command	2-3
Panel content type	Active selection	Narrow (changes on selection)	Clicking different element type	1-2
Scroll position within panel	Panel content	Narrow (content-dependent)	Content reload, panel resize	1-3
Keyboard shortcut binding	Application version	Wide (usually stable across versions)	Software update remapping	3-5

Table I: Flow State Disruption Classification

Disruption Source	Category	Under UI Design Control	Frequency (per hour, dockable)	Frequency (per hour, fixed-panel)	Flow Recovery (minutes)
Accidental panel resize	Cascade	Yes	0.2-0.5	0	2-3
Accidental tab detach	Cascade	Yes	0.1-0.3	0	3-5
Tab proliferation (order shift)	Cascade (micro)	Yes	1.5-3.0	0	0.1-0.3
Panel content replacement (Inspector switch)	Cascade (micro)	Partially	2.0-5.0	1.0-3.0	0.05-0.15
Tooltip/popup occlusion	Focus steal	Partially	1.0-2.0	0.5-1.0	0.05-0.1
Slack/Teams notification	External	No	2.0-6.0	2.0-6.0	2-5
Colleague interruption	External	No	0.3-1.0	0.3-1.0	3-8

Disruption Source	Category	Under UI Design Control	Frequency (per hour, dockable)	Frequency (per hour, fixed-panel)	Flow Recovery (minutes)
Phone notification	External	No	0.5-2.0	0.5-2.0	1-3
Meeting reminder	External	No	0.1-0.3	0.1-0.3	5-15

Table J: Flow Duty Cycle Calculation

Variable	Dockable UI	Fixed-Panel	Source
UI-sourced disruptions per hour	4.8-10.8	1.5-4.0	Table I, sum of UI-controlled rows
External disruptions per hour	2.9-9.3	2.9-9.3	Table I, sum of external rows
Total disruptions per hour	7.7-20.1	4.4-13.3	Sum
Mean interval between disruptions (minutes)	3.0-7.8	4.5-13.6	60 ÷ total disruptions
Mean flow re-entry time (minutes)	2.0-5.0	2.0-5.0	Literature consensus
Effective flow per interval (minutes)	1.0-2.8	2.5-8.6	Interval minus re-entry, floor of zero
Flow duty cycle	13%-72%	44%-86%	Effective flow ÷ interval
Midpoint flow duty cycle	~42%	~65%	—
Flow minutes per 6-hour active day	47-259 (midpoint 151)	158-310 (midpoint 234)	Duty cycle × 360 minutes
Additional flow minutes per day (fixed vs dockable)	—	+51 to +83 (midpoint +83)	Difference
Additional flow hours per year (250 days)	—	+213 to +346 (midpoint +346)	Annualized

Table K: Physical Action Strain Profile

Action Type	Muscles Engaged	Contraction Duration (ms)	Force Level	Strain Classification	Daily Count (Dockable)	Daily Count (Fixed-Panel)
Click (interleave)	Finger flexors (FDP, FDS)	50-100	Low	Repetitive low-strain	600	0
Mouse movement (interleave)	Wrist extensors (ECRL, ECRB), forearm pronator	200-500	Low-medium	Repetitive medium-strain	1,200	0
Drag (cascade recovery)	Finger flexors (sustained), wrist extensors, forearm pronator (simultaneous)	500-2,000	Medium-high	Sustained contraction with movement — highest strain	10-15	0
Eye saccade (reduction scan)	Extraocular muscles	20-50	Negligible (muscular)	Cognitive load, not muscular strain	150-300	0-50
Keyboard shortcut	Finger flexors, finger extensors	50-150	Low	Repetitive low-strain	Same	Same
Productive click	Finger flexors	50-100	Low	Repetitive low-strain (unavoidable)	Same	Same

Action Type	Muscles Engaged	Contraction Duration (ms)	Force Level	Strain Classification	Daily Count (Dockable)	Daily Count (Fixed-Panel)
Productive mouse movement	Wrist extensors, forearm pronator	200-800	Low-medium	Repetitive medium-strain (unavoidable)	Same	Same

Table L: Ten-Year Cumulative Cost Comparison

Cost Category	Dockable UI (decade)	Fixed-Panel (decade)	Difference
Total non-productive operations	1,905,000-2,362,500	0-125,000	1,780,000-2,362,500 eliminated
Interleave operations	1,500,000	0	1,500,000 eliminated
Cascade recovery operations	30,000-112,500	0	30,000-112,500 eliminated
Reduction operations	375,000-750,000	0-125,000	250,000-750,000 eliminated
Cascade events (flow disruptions)	5,000-12,500	0	5,000-12,500 prevented
Unnecessary mouse movements	3,000,000-4,000,000	0	3,000,000-4,000,000 prevented
Sustained-pressure drags	25,000-37,500	0	25,000-37,500 prevented
Dissolved layout elements maintained	75-200 per user	3-8 per user	67-197 freed for domain knowledge
Flow-state hours lost	2,130-3,460	0	2,130-3,460 recovered
Cross-application layout context switches	75,000-125,000	0	75,000-125,000 eliminated

Table M: Application Mapping to Six Panel Areas

Application	Tree	Canvas	Properties	View	Tool Set	Tray
Code Editor	File browser	Editor surface (splittable)	Symbol outline, git status	File path, cursor position	Terminal, build output, problems	Find/replace, minimap
3D Modeling	Scene outliner	3D viewport (quad-splittable)	Object properties, modifiers	Transform coords, pivot mode	Timeline, dopesheet, graph editor	Gizmo controls, snap settings
CAD	Feature tree	Drawing/3D viewport	Dimensions, constraints, material	Coordinates, units, scale	BOM, simulation results	Sketch tools, geometric constraints
Database Admin	Schema browser	Query editor + results (splittable)	Column definitions, indexes	Connection info, row count	Query history, saved queries	Execution status, transaction state
Email	Folder list	Message list + reading pane	Headers, attachments, contact	Current folder, filter state	Compose, calendar	Quick reply, formatting
Spreadsheet	Sheet list, named ranges	Cell grid	Formula bar, cell format	Cell reference, sheet stats	Chart editor, pivot builder	Status bar, zoom
Word Processor	Document outline	Document surface	Paragraph format, styles	Page number, word count	Revision pane, mail merge	Find/replace, spelling
Video Editor	Media bin	Source + program monitors	Clip properties, effects	Timecode, sequence settings	Timeline (multi-track)	Transport controls
DAW	Track list	Arrangement view	Channel strip, plugin params	Meters, tempo, key	Mixer, piano roll	Transport, loop controls
Photo Editor	Layer stack	Image surface	Brush settings, color picker	Navigator, dimensions	Actions, batch processing	Toolbox

Application	Tree	Canvas	Properties	View	Tool Set	Tray
Vector Graphics	Layer/object list	Artboard	Fill/stroke, typography	Artboard selector, rulers	Symbol library, appearance	Pen controls, node editing
Node Graph	Node palette	Graph surface	Node parameters	Output preview	Variable list, compile log	Connection filter, search
Game Engine	Data browser	Game scene viewport	Entity properties	Breadcrumbs, path info	Tool palette	Painting, placement controls
Terminal/3270	Session list	Terminal surface (splittable)	Session properties	Connection status, dimensions	Macro editor, key mapping	Function key bar
File Manager	Directory tree	File list (dual-pane capable)	File properties	Current path, disk usage	Search results, operation queue	Selection status, free space
Diagramming	Shape library	Diagram surface	Shape properties	Page dimensions, grid	Data linking, layers	Connector tools, alignment
GIS	Layer list	Map view	Feature attributes, symbology	Coordinates, projection	Processing toolbox	Digitizing, measurement
Music Notation	Part list	Score surface	Note properties	Measure, key/time signature	Mixer, playback	Note input palette
EDA/PCB	Component library	Schematic/PCB surface	Pin assignments, parameters	Cursor coords, active layer	DRC results, BOM	Routing tools, via placement
Accounting	Chart of accounts	Transaction form, ledger	Account details, tax codes	Current period, entity	Report builder, reconciliation	Calculator, currency
Chat/Messaging	Channel list	Message stream	User card, channel settings	Channel name, topic	Search results, integrations	Formatting, emoji, attachments

Application	Tree	Canvas	Properties	View	Tool Set	Tray
Calendar	Calendar list	Calendar grid	Event details	Current date, range	Agenda, room finder	Quick event, timezone
PDF Viewer	Page thumbnails	Page surface	Annotation properties	Page number, zoom	Comment list, signatures	Annotation toolbar
System Monitor	Process tree	Process list or charts	Process details	System summary	Historical graphs, connections	Kill/suspend buttons, refresh
Log Viewer	Saved searches, field list	Event table	Full event detail	Query, time range, count	Query editor, histogram	Filter buttons, time presets
Package Manager	Category browser	Package detail view	Metadata, dependencies	Filter, search query	Dependency graph, changelog	Install/update buttons
Form Builder	Table/form list	Form design surface	Control properties, events	Data source, form metadata	Controls palette, script editor	Alignment, grid, test data
Presentation	Slide sorter	Current slide	Shape properties, animations	Slide number, notes preview	Speaker notes, animation timeline	Drawing tools

Table N: Three-Axis Configuration Patterns

Pattern	Tool Set Axis	Side Panel Axis	Canvas Axis	Use Case
Unified	Tool A	Tool A	Tool A	Single-tool focused work — editing, modeling, composing
Data + Scene	Data tool	Data Browser	Game Scene	Inspecting database records while viewing game output

Pattern	Tool Set Axis	Side Panel Axis	Canvas Axis	Use Case
Paint + Inspect	Tile Painting	Data Browser	Game Scene	Painting tiles while reading entity properties
Dual Tool	Tool A controls	Tool B navigation/props	Tool A workspace	Using one tool's controls while browsing another tool's data
Monitor + Edit	Monitoring tool	Edit tool	Live view	Watching live system while editing configuration
Reference + Create	Creation tool	Reference browser	Creation workspace	Building new content while browsing reference material

---

Table O: Dissolution Curve for Layout Knowledge

Session Count	Panel Position Ops	Tab Location Ops	Shortcut Ops	Recovery Procedure Ops	Total Layout Ops Per Task Cycle
1 (first session)	3-5 per panel access	2-4 per tab switch	5-8 per shortcut attempt	N/A (no disruption yet)	12-20
5	1-2	1-2	2-3	8-12 (first cascade)	5-10
20	0-1	0-1	0-1	4-6	1-3
50	0	0	0	2-3	0-1
100+	0	0	0	1-2	0 (dissolved)
Post-update (reset)	2-3 (promoted)	1-3 (promoted)	0-1 (usually stable)	6-9 (re-learn if changed)	3-8 (partial cascade)
Fixed-panel session 1	0-1	0	3-5	0	3-6

Session Count	Panel Position Ops	Tab Location Ops	Shortcut Ops	Recovery Procedure Ops	Total Layout Ops Per Task Cycle
Fixed-panel session 5	0	0	0-1	0	0-1
Fixed-panel session 10+	0	0	0	0	0

The fixed-panel system reaches full dissolution in approximately ten sessions. The dockable system reaches full dissolution in approximately one hundred sessions but is subject to periodic cascade-driven re-dissolution that the fixed-panel system never experiences.

Table P: Panel Sizing — The One Retained Capability

Capability	Docking Paradigm	Fixed-Panel + Draggable Edges	Risk Profile
Resize adjacent panels	Yes — drag any splitter	Yes — drag edges between fixed areas	Zero cascade: no panel can detach, float, or reorder
Detach panel to floating window	Yes — drag tab past threshold	No	Cascade source eliminated
Reorder tabs within dock	Yes — drag tab within bar	No	Tab-order dissolution maintained
Split area into new regions	Yes — drag to split target	No — internal subdivision handled by tool commands	Accidental split eliminated
Merge areas	Yes — drag to merge target	No	Accidental merge eliminated
Tear off to OS window	Yes — drag past window edge	No	OS-level window cascade eliminated
Save/restore layout preset	Yes — serialize full dock tree	No — layout is structural constant	Preset mismatch cascade eliminated
Adjust size ratios continuously	Yes	Yes	Identical capability, zero additional risk

---

Table Q: Processing Entropy Profile Comparison (Layout Domain)

Operation	Hp (Dockable, Novice)	Hp (Dockable, Expert)	Hp (Dockable, Post-Cascade)	Hp (Fixed-Panel, Any User)
Locate tree panel	3-5	0	2-3 (promoted)	0 (structural)
Locate properties panel	3-5	0	2-3 (promoted)	0 (structural)
Locate canvas area	1-2	0	0-1	0 (structural)
Find correct tab in properties	3-5	0	1-2 (if tab order changed)	0 (no tabs)
Open tool set bar	2-4	0	0 (shortcut stable)	0 (shortcut stable)
Restore detached panel	8-12	2-3	6-9 (unfamiliar new position)	undefined (cannot occur)
Switch workspace preset	3-5	0-1	2-3 (if presets changed)	undefined (no presets)
Resize panel to preferred width	2-3	0-1	0-1	0-1 (same capability)
Total layout Hp	**25-41**	**2-5**	**13-22**	**0-1**

The fixed-panel system produces a processing entropy profile of zero across all layout operations for all users at all experience levels. The dockable system achieves near-zero only for experts who have not experienced a recent cascade. Post-cascade, even experts regress to novice-range processing entropy for layout operations until re-dissolution completes.

Table R: Specification Summary

Metric	Count
Non-productive operation categories defined	3 (interleave, cascade, reduction) + 3 sub-categories (re-verification, re-dissolution, layout context switch)
Applications analyzed for operation counts	5 (code editor, 3D modeling, database admin, email, game engine)
Applications mapped to six panel areas	28
Three-axis configuration patterns documented	6

Metric	Count
Cascade triggers inventoried	7
Validity envelope dimensions catalogued	10
Flow disruption sources classified	9 (5 UI-controlled, 4 external)
Physical action types profiled	7
Dissolved layout elements per application (dockable)	25-50
Dissolved layout elements per application (fixed-panel)	3-8
Sessions to full layout dissolution (dockable)	~100
Sessions to full layout dissolution (fixed-panel)	~10
Non-productive operations eliminated per day	700-950
Flow-state hours recovered per year	213-346
Unnecessary mouse movements eliminated per decade	3,000,000-4,000,000

HOWL-USER-1-2026 Appendix Tables. Layout Is Not a Feature.

[**HOWL-USER-1-2026**] Layout Is Not a Feature. Github: https://github.com/ghowland/papers/tree/main/papers/USER/HOWL-USER-1-2026

[**HOWL-INFO-14-2026**] Bits and Ops. Github: https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-14-2026

[**HOWL-INFO-11-2026**] The Relationship of Zero, One, and Infinity in Information Processing. Github: https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-11-2026

[**HOWL-INFO-12-2026**] Information Processing Requires Reduction to Cardinality One. Github: https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-12-2026

[**HOWL-INFO-13-2026**] The Six States of Information. Github: https://github.com/ghowland/papers/tree/main/papers/INFO-13-2026

[**HOWL-MATH-16-2026**] The Geometry of Dissolution and Fragility. Github: https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-16-2026

[**HOWL-MATH-18-2026**] Concurrency Tax from System Structure. Github:

https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-18-2026